

Figure 1: Architecture of SBC OS

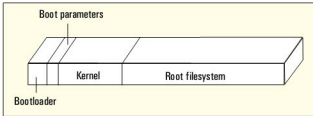


Figure 2: Storage of the boot loader, kernel and root file system in the ROM memory (Flash)

not hold an operating system, so special software is needed to bring the SBC OS into the memory from the media on which it resides. This software is normally a small piece of code called the boot loader. On a desktop PC, the boot loader resides on the master boot record (MBR) of the hard drive and is executed after the PC's basic input output system (BIOS) performs system initialisation tasks. In a SBC, the boot loader's role is more complicated because these systems rarely have a BIOS to perform initial system configuration.

At the very minimum, a boot loader for the SBC OS performs these functions:

- Initialises the hardware, especially the memory controller
- Provides boot parameters for the SBC OS
- Starts the SBC OS

Compiling the boot loader

Boot loader compilation for the SBC OS can be done using the following command:

```
$ make clean && make cubietruck CROSS_COMPILE=arm-linux-gnueabi-
```

Building the kernel – ulmage

The kernel is the most fundamental software component of the SBC OS. It is responsible for managing the bare hardware within the chosen target system and bringing order to what would otherwise be a chaotic struggle between each of the many different software components on a typical system.

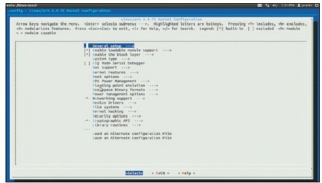


Figure 3: Snapshot of Menuconfig

Configuring the kernel

The options that we need in the kernel have to be configured before building it. The target is to have an appropriate `.config` file in our Kernel source distribution. Depending on our target, the option menus available will change, as will their content. Some options, however, will be available no matter which embedded architecture we choose. After the environmental set-up, `make menuconfig` runs a text-based menu interface as shown in Figure 3.

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- menuconfig
```

<> indicates that the feature is on

<M> indicates that the feature is configured as module

<> indicates that the feature is off

Compiling the kernel and modules

After saving the kernel configuration in the kernel root directory, our main goals are to compile the ulmage compressed kernel and ulmage kernel modules for the SBC OS using the following commands:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- uImage -j4
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules -j4
```

Installing modules

Before we install and boot from our new SBC OS kernel, we should put the new kernel modules in `/lib/modules` with the following command:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules_install
```

Our new modules should appear in `/lib/modules/version`, where the version is the new kernel version of the SBC OS.

Designing the root file system

One of the last operations conducted by the Linux kernel during system start-up is mounting the root file system. The Linux kernel itself doesn't dictate any file system structure, but user space applications do expect to find files with